Linear Data Structures

沈威宇

November 8, 2026

1 Linear Data Structures

I Unordered list

An unordered list is a list where each element holds a relative position with respect to the others. Each element is called a node. If applicable, the first node is called head and the last node is called tail.

II Ordered list

An ordered list is a list which are ordered based upon some underlying characteristic of the elements.

III Stack

A stack is a last-in-first-out (LIFO) list. Adding an element is called push and is of time complexity O(1), the last pushed element is called top, and the first pushed element is called bottom. Removing and returning top is called pop and is of time complexity O(1). Peeking top is called top and is of time complexity O(1).

To keep track of the minimum element of a stack in O(1) time with O(n) extra space. One can maintain the original stack of elements and an extra stack of extrema. When an element is pushed to the original stack, it is compared with the top of the extrema stack and the minimum one is pushed to the extrema stack (for the first element, it is pushed to the extrema stack). When an element is popped from the original stack, an element is popped from the extrema stack. This way, the top of the extrema stack is always the minimum of the elements in the original stack.

IV Queue

A queue is first-in-first-out (FIFO) list. Adding an element is called enqueue and is of time complexity O(1), the first enqueued element is called front, and the last enqueued element is called back. Removing and returning front is called dequeue and is of time complexity O(1). Peeking front is called peek and is of time complexity O(1).

V Array

An array is a fixed-size list of elements of the same memory size, each identified by at least one index, stored contiguously such that the position (memory address) of each element can be computed from its index in O(1) time, i.e., random access in O(1) time.

VI Dynamic array, growable array, resizable array, dynamic table, mutable array, or array list

A dynamic array is a variable-size list that stores elements in an array of a capacity and reserves an array of new capacity elsewhere and moves to it whenever its size reaches its capacity. It has random access and popping back in O(1) time, pushing back in amortized O(1) time, but removal and insertion in O(n) time. It is a stack with front as bottom and back as top.

VII Singly linked list

A singly linked list is an unordered list where each node stores only the next node. It has O(1) peeking, pushing, and popping front and insertion after node in O(1) time, but random access, removal of node, and insertion before node in O(n) time. If the tail is stored, it has peeking and pushing back in O(1) time. It is a stack with head as top and tail as bottom. If the tail is stored, it is a queue with the tail as front and head as back.

An implementation in Python:

```
class _SNode:
    def __init__(self, data, next=None):
        self.data = data
        self.next = next


class SinglyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def __len__(self):  # O(1)
        return self.size

    def is_empty(self):  # O(1)
        return self.head is None

    def __getitem__(self, num):  # O(num)
        if not 0 <= num < self.size:
            raise IndexError("list index out of range")
        cur = self.head
        for _ in range(0, num):
            cur = cur.next
        return cur.data

    def __setitem__(self, num, x):  # O(num)
        if not 0 <= num < self.size:
            raise IndexError("list index out of range")
        cur = self.head
        for _ in range(0, num):
            cur = cur.next
        cur.data = x

    def push_front(self, x):  # O(1)
        self.head = _SNode(x, self.head)
```

```
        if self.tail is None:
            self.tail = self.head
        self.size += 1

    def push_back(self, x):  # O(1)
        node = _SNode(x)
        if self.tail is None:
            self.head = self.tail = node
        else:
            self.tail.next = node
            self.tail = node
        self.size += 1

    def pop_front(self):  # O(1)
        if self.head is None:
            raise IndexError("remove from empty list")
        x = self.head.data
        self.head = self.head.next
        self.size -= 1
        if self.head is None:
            self.tail = None
        return x

    def peek_front(self):  # O(1)
        if self.is_empty():
            raise IndexError("peek at empty list")
        return self.head.data

    def peek_back(self):  # O(1)
        if self.is_empty():
            raise IndexError("peek at empty list")
        return self.tail.data

    def insert_after_node(self, pos, x):  # O(1)
        if self.is_empty():
            raise IndexError("insert at empty list")
        if pos is self.tail:
            return self.push_back(x)
        node = _SNode(x, pos.next)
        pos.next = node
        self.size += 1

    def clear(self):  # O(1)
        self.head = None
        self.tail = None
        self.size = 0

    def __iter__(self):
        cur = self.head
        while cur is not None:
            yield cur.data
            cur = cur.next
```

VIII Doubly linked list

A doubly linked list is an unordered list where each node stores the previous node and the next node. It has peeking, pushing, and popping front and back, insertion after and before node, and removal of node in O(1) time, but random access in O(n) time. It is a

deque.

An implementation in Python:

```
class _DNode:
    def __init__(self, data, prev=None, next=None):
        self.data = data
        self.prev = prev
        self.next = next


class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def __len__(self):  # O(1)
        return self.size

    def is_empty(self):  # O(1)
        return self.size == 0

    def __getitem__(self, num):  # O(min(n, size - n)), n = num if num > 0 else num + size
        if not -self.size <= num < self.size:
            raise IndexError("list index out of range")
        if num * 2 >= self.size:
            num -= self.size
        elif num * 2 < -self.size:
            num += self.size
        if num >= 0:
            cur = self.head
            for _ in range(0, num):
                cur = cur.next
        else:
            cur = self.tail
            for _ in range(-1, num, -1):
                cur = cur.prev
        return cur.data

    def __setitem__(self, num, x):  # O(min(n, size - n)), n = num if num > 0 else num + size
        if not -self.size <= num < self.size:
            raise IndexError("list index out of range")
        if num * 2 >= self.size:
            num -= self.size
        elif num * 2 < -self.size:
            num += self.size
        if num >= 0:
            cur = self.head
            for _ in range(0, num):
                cur = cur.next
        else:
            cur = self.tail
            for _ in range(-1, num, -1):
                cur = cur.prev
        cur.data = x

    def push_front(self, x):  # O(1)
        node = _DNode(x, None, self.head)
        if self.head is None:
            self.head = self.tail = node
        else:
```

```
            self.head.prev = node
            self.head = node
        self.size += 1

    def push_back(self, x):  # O(1)
        node = _DNode(x, self.tail, None)
        if self.tail is None:
            self.head = self.tail = node
        else:
            self.tail.next = node
            self.tail = node
        self.size += 1

    def pop_front(self):  # O(1)
        if self.head is None:
            raise IndexError("pop from empty list")
        x = self.head.data
        self.head = self.head.next
        self.size -= 1
        if self.head is None:
            self.tail = None
        else:
            self.head.prev = None
        return x

    def pop_back(self):  # O(1)
        if self.tail is None:
            raise IndexError("pop from empty list")
        x = self.tail.data
        self.tail = self.tail.prev
        self.size -= 1
        if self.tail is None:
            self.head = None
        else:
            self.tail.next = None
        return x

    def peek_front(self):  # O(1)
        if self.is_empty():
            raise IndexError("peek at empty list")
        return self.head.data

    def peek_back(self):  # O(1)
        if self.is_empty():
            raise IndexError("peek at empty list")
        return self.tail.data

    def insert_after_node(self, pos, x):  # O(1)
        if self.is_empty():
            raise IndexError("insert at empty list")
        if pos is self.tail:
            return self.push_back(x)
        node = _DNode(x, pos, pos.next)
        pos.next.prev = node
        pos.next = node
        self.size += 1

    def insert_before_node(self, pos, x):  # O(1)
        if self.is_empty():
            raise IndexError("insert at empty list")
        if pos is self.head:
            return self.push_front(x)
```

```
        node = _DNode(x, pos.prev, pos)
        pos.prev.next = node
        pos.prev = node
        self.size += 1

    def remove_node(self, node):  # O(1)
        if self.head is None:
            raise IndexError("remove from empty list")
        if self.head is node:
            return self.pop_front()
        elif self.tail is node:
            return self.pop_back()
        node.prev.next = node.next
        node.next.prev = node.prev
        self.size -= 1
        return node.data

    def clear(self):  # O(1)
        self.head = None
        self.tail = None
        self.size = 0

    def __iter__(self):
        cur = self.head
        while cur is not None:
            yield cur.data
            cur = cur.next

    def __reversed__(self):
        cur = self.tail
        while cur is not None:
            yield cur.data
            cur = cur.prev
```

IX Deque

A doubly-ended queue (deque) is a list that is simultaneously a stack and a queue, that is, has peeking, pushing, and popping from both ends in O(1) time.

An implementation of deque is a dynamic array of fixed-size blocks, which has random access and popping front and back in O(1) time, pushing front and back in amortized O(1) time, but removal and insertion in O(n) time. A such implementation in Python:

```
class BlockListDeque:  # Both ends IO amortized O(1)
    def __init__(self, block_size=64, front_step=1, back_step=1):
        if block_size <= 0:
            raise ValueError("deque block_size must >0")
        if front_step <= 0:
            raise ValueError("deque front_step must >0")
        if back_step <= 0:
            raise ValueError("deque back_step must >0")
        self.block_size = block_size
        self.front_step = front_step
        self.back_step = back_step
        self.data = [[None] * self.block_size for _ in range(self.front_step + self.back_step - 1)]
        self.block_begin = self.front_step - 1
        self.block_end = self.block_begin + 1
        self.in_begin = (self.block_size - 1) // 2
```

```
        self.in_end = self.in_begin
        self.size = 0

    def __len__(self):  # O(1)
        return self.size

    def is_empty(self):  # O(1)
        return self.size == 0

    def capacity(self):  # O(1)
        return self.block_size * len(self.data)

    def is_valid(self):  # O(1)
        return self.size == self.block_size * (self.block_end - self.block_begin - 1) + self.in_end
            - self.in_begin

    def __getitem__(self, index):  # O(1)
        if not -self.size <= index < self.size:
            raise IndexError("deque index out of range")
        if index < 0:
            index += self.size
        block_offset, in_block = divmod(
            self.in_begin + index,
            self.block_size,
        )
        return self.data[self.block_begin + block_offset][in_block]

    def __setitem__(self, index, x):  # O(1)
        if not -self.size <= index < self.size:
            raise IndexError("deque index out of range")
        if index < 0:
            index += self.size
        block_offset, in_block = divmod(
            self.in_begin + index,
            self.block_size,
        )
        self.data[self.block_begin + block_offset][in_block] = x

    def reserve_front(self, blocks=None):  # O(1)
        if blocks is None:
            blocks = self.front_step
        if blocks <= self.block_begin:
            return
        new = [[None] * self.block_size for _ in range(blocks)]
        new.extend(self.data)
        self.data = new
        self.block_begin += blocks
        self.block_end += blocks

    def reserve_back(self, blocks=None):  # O(1)
        if blocks is None:
            blocks = self.back_step
        if self.block_end + blocks <= len(self.data):
            return
        self.data.extend([[None] * self.block_size for _ in range(blocks)])

    def shrink_front_to_fit(self):
        new = self.data[self.block_begin :]
        self.data = new
        self.block_end -= self.block_begin
        self.block_begin = 0

```

```
    def shrink_back_to_fit(self):
        for _ in range(self.block_end, len(self.data)):
            self.data.pop()

    def push_front(self, x):  # amortized O(1)
        if self.in_begin != 0:
            self.in_begin -= 1
        else:
            if self.block_begin == 0:
                self.reserve_front()
            self.block_begin -= 1
            self.in_begin = self.block_size - 1
        self.data[self.block_begin][self.in_begin] = x
        self.size += 1

    def push_back(self, x):  # amortized O(1)
        if self.in_end != self.block_size:
            self.in_end += 1
        else:
            if self.block_end == len(self.data):
                self.reserve_back()
            self.block_end += 1
            self.in_end = 1
        self.data[self.block_end - 1][self.in_end - 1] = x
        self.size += 1

    def pop_front(self):  # O(1)
        if self.is_empty():
            raise IndexError("pop from empty deque")
        x = self.data[self.block_begin][self.in_begin]
        self.data[self.block_begin][self.in_begin] = None
        self.size -= 1
        if self.in_begin != self.block_size - 1:
            self.in_begin += 1
        else:
            self.block_begin += 1
            self.in_begin = 0
        return x

    def pop_back(self):  # O(1)
        if self.is_empty():
            raise IndexError("pop from empty deque")
        x = self.data[self.block_end - 1][self.in_end - 1]
        self.data[self.block_end - 1][self.in_end - 1] = None
        self.size -= 1
        if self.in_end != 1:
            self.in_end -= 1
        else:
            self.block_end -= 1
            self.in_end = self.block_size
        return x

    def peek_front(self):  # O(1)
        if self.is_empty():
            raise IndexError("peek from empty deque")
        return self.data[self.block_begin][self.in_begin]

    def peek_back(self):  # O(1)
        if self.is_empty():
            raise IndexError("peek from empty deque")
        return self.data[self.block_end - 1][self.in_end - 1]

```

```
    def clear(self):  # O(len(self.data))
        num_blocks = len(self.data)
        self.data = [[None] * self.block_size for _ in range(num_blocks)]
        self.block_begin = self.front_step - 1
        self.block_end = self.block_begin + 1
        self.in_begin = (self.block_size - 1) // 2
        self.in_end = self.in_begin
        self.size = 0

    def __iter__(self):
        block = self.block_begin
        index = self.in_begin
        for _ in range(self.size):
            yield self.data[block][index]
            index += 1
            if index == self.block_size:
                index = 0
                block += 1

    def __reversed__(self):
        block = self.block_end - 1
        index = self.in_end - 1
        for _ in range(self.size):
            yield self.data[block][index]
            index -= 1
            if index < 0:
                index = self.block_size - 1
                block -= 1
```